

ALGORITHMS 1 – LAB

The Binary Search Tree ADT

MARCO GUAZZONE

DiSIT, University of Piemonte Orientale
`marco.guazzone@uniupo.it`

The Binary Search Tree ADT

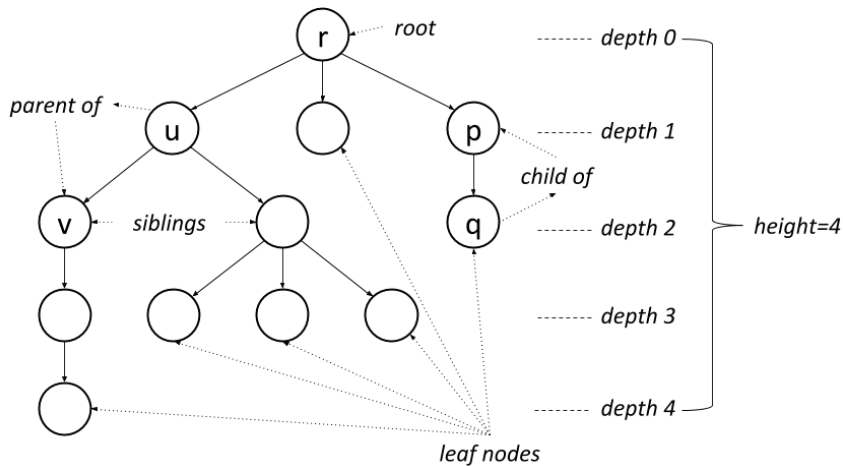
A **Tree** is a container where elements are organized according to a hierarchical structure

- Elements are connected with each other by a *parent-child* relationship



The Binary Search Tree ADT

Definitions



The Binary Search Tree ADT

Definitions (cont'd)

Binary Trees are a special case of Trees where each node can have at most two children

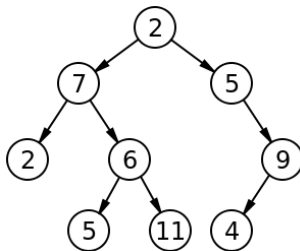


Image source: "Binary tree" by Derrick Coetzee – Own work. Licensed under Public Domain via Commons –

https://commons.wikimedia.org/wiki/File:Binary_tree.svg#/media/File:Binary_tree.svg

The Binary Search Tree ADT

Definitions (cont'd)

The five possible binary trees with three nodes:

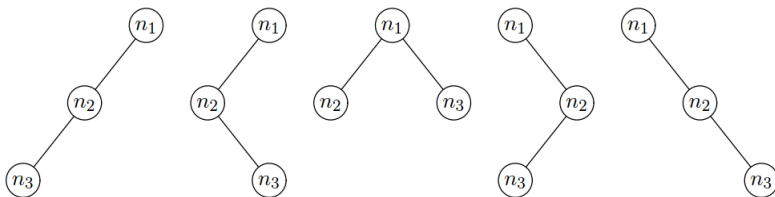


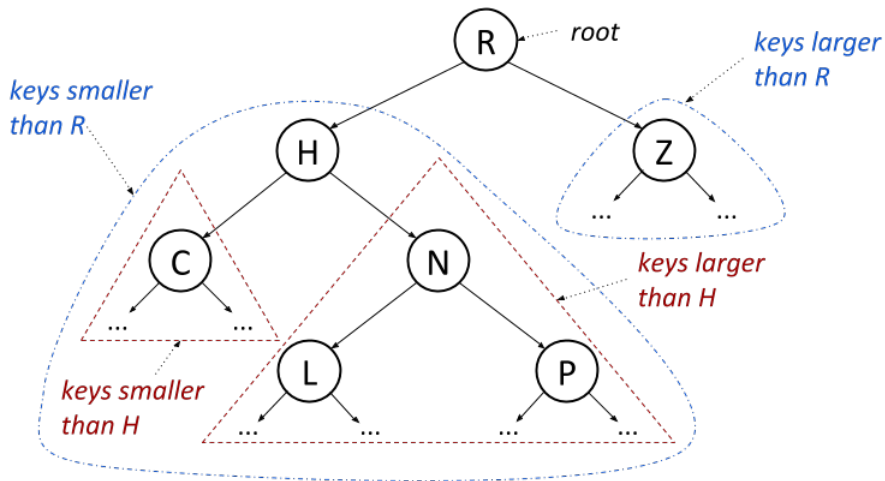
Image source: A.V. Aho et al., "Foundation of Computer Science, C Version," Prentice-Hall, 1994

The Binary Search Tree ADT

A **Binary Search Tree** (BST) is a binary tree where:

- Each node has a **key** and an associated **value** (possibly, the key itself),
- The following property (**BST property**) holds:
 - The key in node v is greater than the keys in all nodes in the left subtree of v , and
 - The key in node v is less than the keys in all nodes in the right subtree of v

The Binary Search Tree ADT



NOTE: for each node, only the key is shown.

The Binary Search Tree ADT

Practical applications

- Searching problems
- Adaptive Sorting: after each insertion, the set of elements seen so far is available in sorted order
 - *Tree sort* is a quadratic sorting algorithm that builds a BST from the keys to be sorted, and then traverses the tree in-order
- Implementation of different ADTs
 - Sets
 - Dictionaries
 - Priority Queues
- ...

Fundamental Operations

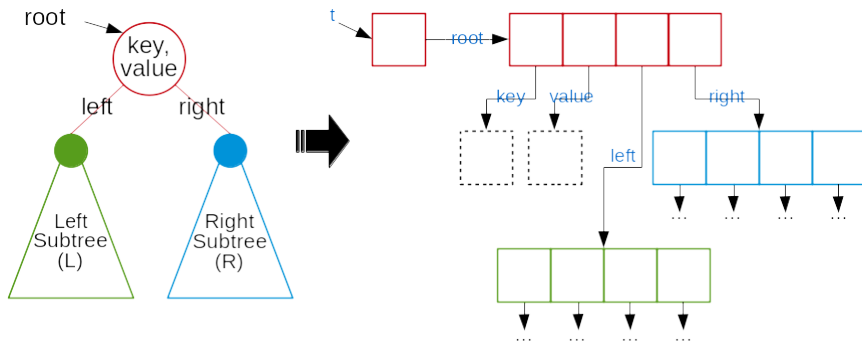
A BST is characterized by the following operations:

- `get( $t, k$ )`: returns the value v identified by k in t
- `contains( $t, k$ )`: tells if a value identified by k exists in t
- `put( $t, k, v$ )`: inserts/replaces the value v identified by k in t and returns the previous value (if any)
- `insert( $t, k, v$ )`: inserts the value v identified by k in t only if it isn't a duplicate
- `delete( $t, k$ )`: removes the key-value pair identified by k from t
- `clear( $t$ )`: removes all the key-value pairs from t
- `is_empty( $t$ )`: tells whether t is an empty tree or not
- `size( $t$ )`: returns the number of nodes in t
- `height( $t$ )`: returns the height of t
- `traverse_in_order( $t, v$ )`: performs a (depth-first) in-order traversal of t , visiting each node with function v

The Binary Search Tree ADT

Fundamental Operations (cont'd)

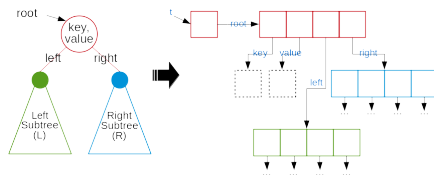
Pointers-to-children representation of BSTs:



Fundamental Operations (cont'd)

Let t be a BST and n a node of the BST,

- $\text{Root}(t)$: the root node of the tree t
- $\text{Key}(n)$: the key of the node n
- $\text{Value}(n)$: the value of the node n
- $\text{Left}(n)$: the left child of the node n
- $\text{Right}(n)$: the right child of the node n



EXAMPLE: $\text{Key}(\text{Left}(\text{Root}(t)))$

The key of the left-child node of the root of the tree t

Fundamental Operations (cont'd)

Most of the fundamental operations can be implemented **recursively**

- Recursive calls operate on nodes
- But public operations take as argument a tree (which is not a node)

→ **Public** operation takes a tree and calls a **private** recursive implementation that takes a node

EXAMPLE: The `get(t,k)` operation

```
1: function GET(t, k)
2:   /* Public non-recursive
   operation */
3:   n ← GETIMPL(ROOT(t), k)
4:   ...
5: end function
```

```
1: function GETIMPL(n, k)
2:   /* Private recursive
   implementation */
3:   ...
4: end function
```

Fundamental Operations (cont'd)

The $\text{get}(t, k)$ operation: searches a key and returns the associated value

```
1: function GET( $t, k$ )
2:    $n \leftarrow \text{GETIMPL}(\text{ROOT}(t), k)$ 
3:   if  $n \neq \text{null}$  then
4:     return VALUE( $n$ )
5:   else
6:     return null
7:   end if
8: end function
```

```
1: function GETIMPL( $n, k$ )
2:   if  $n = \text{null}$  then
3:     return null
4:   end if
5:   if  $k < \text{KEY}(n)$  then
6:     return GETIMPL(LEFT( $n$ ),  $k$ )
7:   else if  $k > \text{KEY}(n)$  then
8:     return GETIMPL(RIGHT( $n$ ),  $k$ )
9:   else
10:    return  $n$ 
11:  end if
12: end function
```

Fundamental Operations (cont'd)

The `contains(t,k)` operation: like the `get(t,k)` operation, but returns true or false

```
1: function CONTAINS( $t, k$ )  
2:   if GETIMPL(ROOT( $t$ ),  $k$ )  $\neq$  null then  
3:     return true  
4:   else  
5:     return false  
6:   end if  
7: end function
```

The Binary Search Tree ADT

Fundamental Operations (cont'd)

The $\text{put}(t, k, v)$ operation: inserts a new key-value pair or updates the value of an existing key, and returns the previously stored value (in case of an update) or `null` (in case of an insertion)

key value

S

0



A

2



*black nodes
are accessed
in search*

E

1



R

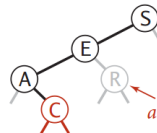
3



*red nodes
are new*

C

4



*gray nodes
are untouched*

Image source: R. Sedgwick et al. "Algorithms, 4/E," Pearson, 2011.

Fundamental Operations (cont'd)

The $\text{put}(t, k, v)$ operation... (cont'd)

```
1: function PUT( $t, k, v$ )  
2:    $v_{old} \leftarrow \text{null}$   
3:    $\text{ROOT}(t) \leftarrow \text{PUTIMPL}(\text{ROOT}(t), k, v, v_{old})$   
4:   return  $v_{old}$   
5: end function
```

```
1: function PUTIMPL( $n, k, v, v_{old}$ )  
2:    $v_{old} \leftarrow \text{null}$   
3:   if  $n = \text{null}$  then  
4:     return  $\text{NODECREATE}(k, v)$   
5:   else if  $k < \text{KEY}(n)$  then  
6:      $\text{LEFT}(n) \leftarrow \text{PUTIMPL}(\text{LEFT}(n), k, v, v_{old})$   
7:   else if  $k > \text{KEY}(n)$  then  
8:      $\text{RIGHT}(n) \leftarrow \text{PUTIMPL}(\text{RIGHT}(n), k, v, v_{old})$   
9:   else  
10:     $v_{old} \leftarrow \text{VALUE}(n)$   
11:     $\text{VALUE}(n) \leftarrow v$   
12:  end if  
13:  return  $n$   
14: end function
```


Fundamental Operations (cont'd)

The $\text{insert}(t, k, v)$ operation: like the $\text{put}(t, k, v)$ operation, but ignores duplicates

```
1: procedure INSERT( $t, k, v$ )  
2:    $\text{ROOT}(t) \leftarrow \text{INSERTIMPL}(\text{ROOT}(t), k, v)$   
3: end procedure
```

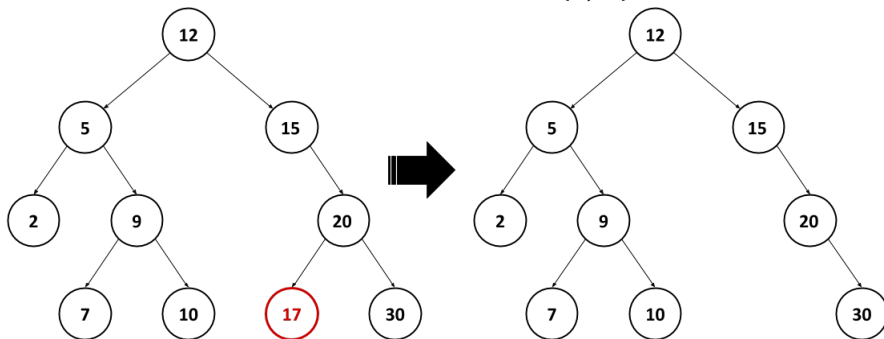
```
1: function INSERTIMPL( $n, k, v$ )  
2:   if  $n = \text{null}$  then  
3:     return NODECREATE( $k, v$ )  
4:   else if  $k < \text{KEY}(n)$  then  
5:      $\text{LEFT}(n) \leftarrow \text{INSERTIMPL}(\text{LEFT}(n), k, v)$   
6:   else if  $k > \text{KEY}(n)$  then  
7:      $\text{RIGHT}(n) \leftarrow \text{INSERTIMPL}(\text{RIGHT}(n), k, v)$   
8:   else  
9:     /* Duplicate key: do nothing */  
10:  end if  
11:  return  $n$   
12: end function
```

The Binary Search Tree ADT

Fundamental Operations (cont'd)

The $\text{delete}(t, k)$ operation: removes a key and the associated value

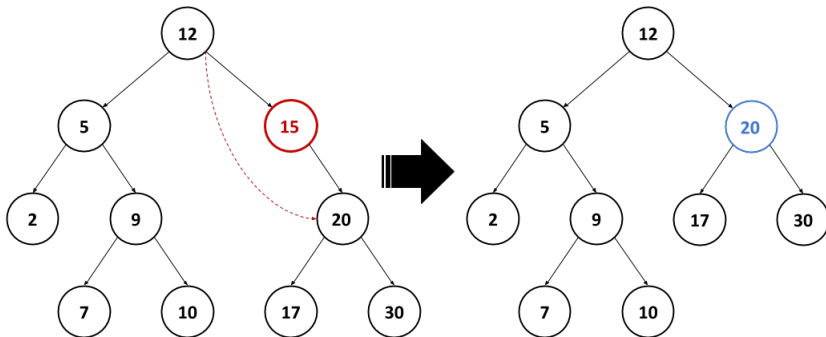
Deletion of a leaf node: $\text{delete}(t, 17)$



Fundamental Operations (cont'd)

The `delete(t,k)` operation... (cont'd)

Deletion of a node with one child: `delete(t, 15)`

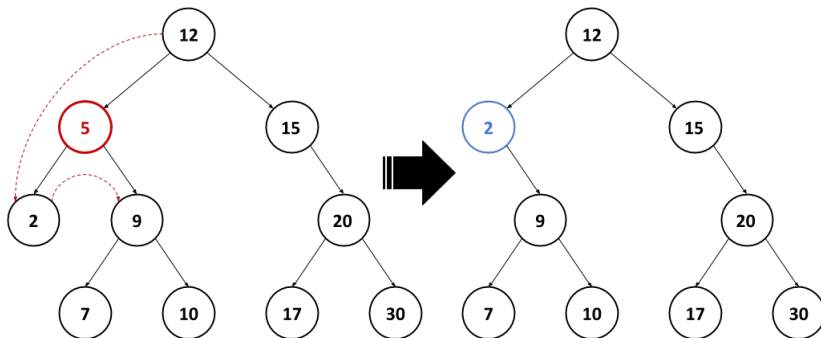


Fundamental Operations (cont'd)

The $\text{delete}(t, k)$ operation... (cont'd)

Deletion of a node with two children: $\text{delete}(t, 5)$

Alternative #1: use the largest predecessor



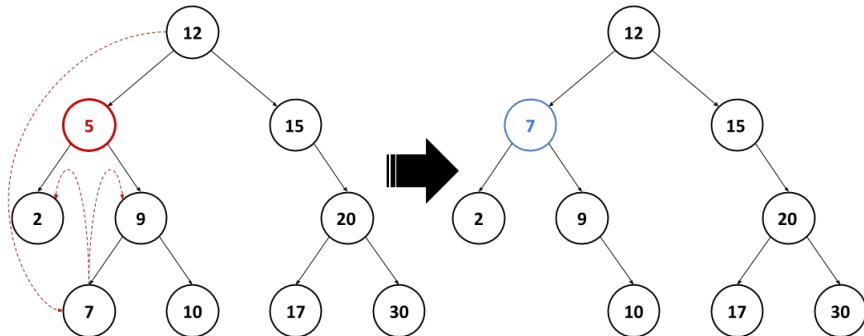
The Binary Search Tree ADT

Fundamental Operations (cont'd)

The $\text{delete}(t, k)$ operation... (cont'd)

Deletion of a node with two children: $\text{delete}(t, 5)$

Alternative #2: use the smallest successor



Fundamental Operations (cont'd)

The delete(t, k) operation... (cont'd)

```
1: procedure DELETE( $t, k$ )  
2:    $\text{Root}(t) \leftarrow$   
   DELETEIMPL( $\text{Root}(t), k$ )  
3: end procedure
```

```
1: function DELETEIMPL( $n, k$ )  
2:   if  $n = \text{null}$  then  
3:     return null  
4:   end if  
5:   if  $k < \text{KEY}(n)$  then  
6:      $\text{LEFT}(n) \leftarrow \text{DELETEIMPL}(\text{LEFT}(n), k)$   
7:   else if  $k > \text{KEY}(n)$  then  
8:      $\text{RIGHT}(n) \leftarrow \text{DELETEIMPL}(\text{RIGHT}(n), k)$   
9:   else if  $\text{LEFT}(n) \neq \text{null}$  and  $\text{RIGHT}(n) \neq \text{null}$  then  
10:     $n \leftarrow \text{DELETE2CIMPL}(n)$   
11:  else  
12:     $n \leftarrow \text{DELETE1CIMPL}(n)$   
13:  end if  
14:  return  $n$   
15: end function
```

Fundamental Operations (cont'd)

The delete(t, k) operation... (cont'd)

```
1: function DELETE1CIMPL( $n$ )
2:    $x \leftarrow n$ 
3:   if LEFT( $n$ )  $\neq$  null then
4:      $n \leftarrow$  LEFT( $n$ )
5:   else
6:      $n \leftarrow$  RIGHT( $n$ )
7:   end if
8:   DESTROYNODE( $x$ )
9:   return  $n$ 
10: end function
```

Alternative #1 (largest predecessor):

```
1: function DELETE2CIMPL( $n$ )
2:    $m \leftarrow$  MAXIMPL(LEFT( $n$ ))
3:   KEY( $n$ )  $\leftarrow$  KEY( $m$ )
4:   VALUE( $n$ )  $\leftarrow$  VALUE( $m$ )
5:   LEFT( $n$ )  $\leftarrow$  DELETEIMPL(LEFT( $n$ ), KEY( $m$ ))
6:   return  $n$ 
7: end function
```

Alternative #2 (smallest successor):

```
1: function DELETE2CIMPL( $n$ )
2:    $m \leftarrow$  MINIMPL(RIGHT( $n$ ))
3:   KEY( $n$ )  $\leftarrow$  KEY( $m$ )
4:   VALUE( $n$ )  $\leftarrow$  VALUE( $m$ )
5:   RIGHT( $n$ )  $\leftarrow$  DELETEIMPL(RIGHT( $n$ ), KEY( $m$ ))
6:   return  $n$ 
7: end function
```

Fundamental Operations (cont'd)

The `delete(t,k)` operation... (cont'd)

For Alternative #1:

```
1: function MAXIMPL(n)
2:   if n = null then
3:     return null
4:   else if RIGHT(n) ≠ null then
5:     return MAXIMPL(RIGHT(n))
6:   else
7:     return n
8:   end if
9: end function
```

For Alternative #2:

```
1: function MINIMPL(n)
2:   if n = null then
3:     return null
4:   else if LEFT(n) ≠ null then
5:     return MINIMPL(LEFT(n))
6:   else
7:     return n
8:   end if
9: end function
```


Fundamental Operations (cont'd)

The `is_empty(t)` operation: returns `true` if the tree is empty, or `false` otherwise

```
1: function IsEmpty(t)  
2:   if t = null or Root(t) = null then  
3:     return true  
4:   else  
5:     return false  
6:   end if  
7: end function
```

Fundamental Operations (cont'd)

The `size(t)` operation: returns the number of nodes

```
1: function SIZE(t)  
2:   return SIZEIMPL(ROOT(t))  
3: end function
```

```
1: function SIZEIMPL(n)  
2:   if n = null then  
3:     return 0  
4:   end if  
5:   return 1 + SIZEIMPL(LEFT(n)) +  
        SIZEIMPL(RIGHT(n))  
6: end function
```

Fundamental Operations (cont'd)

The `height(t)` operation: returns the height of the tree (i.e., the length of the longest path from root to a leaf)

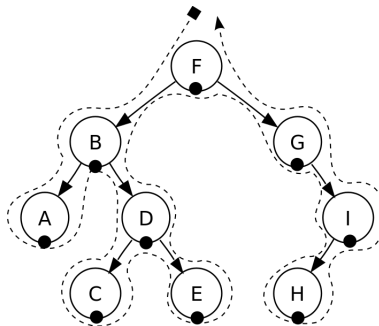
```
1: function HEIGHT(t)  
2:   return HEIGHTIMPL(ROOT(t))  
3: end function
```

```
1: function HEIGHTIMPL(n)  
2:   if n = null or ISLEAFIMPL(n) then  
3:     return 0  
4:   end if  
5:   return 1 + MAX(HEIGHTIMPL(LEFT(n)) ,  
                   HEIGHTIMPL(RIGHT(n)))  
6: end function  
  
1: function ISLEAFIMPL(n)  
2:   if LEFT(n) = null and RIGHT(n) = null then  
3:     return true  
4:   end if  
5:   return false  
6: end function
```

The Binary Search Tree ADT

Fundamental Operations (cont'd)

The `traverse_in_order(t,v)` operation: performs a (depth-first) in-order traversal of the tree, by calling for each node the function `v`



In-order: $v(A)$, $v(B)$, $v(C)$, $v(D)$, $v(E)$, $v(F)$, $v(G)$, $v(H)$, $v(I)$

Image source: "Sorted binary tree inorder" by Pluke. Licensed under Public Domain via Commons –

Fundamental Operations (cont'd)

The `traverse_in_order(t,v)` operation... (cont'd)

```
1: procedure TRAVERSEINORDER( $t, v$ )  
2:   TRAVERSEINORDERIMPL(ROOT( $t$ ),  $v$ )  
3: end procedure
```

```
1: procedure TRAVERSEINORDERIMPL( $n, v$ )  
2:   if  $n \neq \text{null}$  then  
3:     TRAVERSEINORDERIMPL(LEFT( $n$ ),  $v$ )  
4:      $v(n)$   
5:     TRAVERSEINORDERIMPL(RIGHT( $n$ ),  $v$ )  
6:   end if  
7: end procedure
```

The Binary Search Tree ADT `upo_bst_t`

Implementation requirements:

- Keys and values must be of generic type
→ Use `void*`
- To guarantee and check the BST property, we need a *key comparison function*:

```
int compare(const void *a, const void *b);
```

which takes two parameters *a* and *b*, and returns a value:

- < 0 if $*a < *b$
- $= 0$ if $*a == *b$
- > 0 if $*a > *b$

NOTE: $*a, *b$ is only a shorthand notation since `void*` cannot be dereferenced

→ Use a **comparator** (a pointer to a comparison function):

```
typedef int (*upo_bst_comparator_t)(const void*, const void*);
```

and store it in the BST data structure

The Binary Search Tree ADT `upo_bst_t` (cont'd)

Pointers-to-children implementation of BSTs in C:

```
1  /// Type for key comparators
2  typedef int (*upo_bst_comparator_t)(const void*, const void*);
3
4  /// Type for nodes of a BST
5  typedef struct upo_bst_node_s upo_bst_node_t;
6  struct upo_bst_node_s {
7      void *key; // Pointer to the key.
8      void *value; // Pointer to the value associated to the key.
9      upo_bst_node_t *left; // Pointer to the left child node.
10     upo_bst_node_t *right; // Pointer to the right child node.
11 };
12
13 /// Type for BSTs
14 struct upo_bst_s {
15     upo_bst_node_t *root; // Pointer to root node of the BST.
16     upo_bst_comparator_t key_cmp; // The key comparator.
17 };
18 typedef struct upo_bst_s* upo_bst_t;
```

The Binary Search Tree ADT `upo_bst_t` (cont'd)

Additional implementation requirements for element-removal and destroy operations:

- When an element is removed or the ADT is destroyed, who takes care of freeing memory allocated for user data?
→ Use an additional parameter `destroy_data` to let the user decide:

```
void upo_bst_clear(..., int destroy_data);  
void upo_bst_destroy(..., int destroy_data);  
void upo_bst_delete(..., int destroy_data);  
void upo_bst_delete_min(..., int destroy_data);  
void upo_bst_delete_max(..., int destroy_data);
```

If `destroy_data` is $\neq 0$ the memory allocated for user-data (i.e., pointers to keys and values) is deallocated (with the `free()` function)

The Binary Search Tree ADT `upo_bst_t` (cont'd)

Additional implementation requirements for traversal operations:

- To visit each node we need to specify a *node visit function*

```
void visit(void *key, void *value, void *context);
```

which takes three parameters:

- ❶ key is a pointer to the key stored in the node being visited
- ❷ value is a pointer to the value associated to the key
- ❸ context is a pointer to additional information used by the visit function (may be NULL)

→ Use a **visitor** (a pointer to a node visit function):

```
typedef void (*upo_bst_visitor_t)(void*, void*, void*);
```

and pass it as a parameter to the traversal functions

The Binary Search Tree ADT `upo_bst_t` (cont'd)

Functions for the (depth-first) in-order traversal of BSTs:

```
void upo_bst_traverse_in_order(const upo_bst_t tree,  
    upo_bst_visitor_t visit, void *visit_context);
```

where:

- `tree` is the BST to be visited
- `visit` is the node visit function
- `visit_context` is passed as third argument to the node visit function each time it is called by the traversal function:

```
1  // 'node' is the current node being visited by the traversal  
   function  
2  ...  
3  visit(node->key, node->value, visit_context);
```

The Binary Search Tree ADT `upo_bst_t` (cont'd)

EXAMPLE: Print nodes sorted by their keys

```
1 void node_print_visit(void *key, void *value, void *context)
2 {
3     /* Assume keys and values are integer numbers */
4     printf("key: %d -> value: %d\n", *((int*) key), *((int*) value));
5 }
6
7 int main()
8 {
9     upo_bst_t tree;
10    ...
11    upo_bst_traverse_in_order(tree, node_print_visit, NULL);
12    ...
13 }
```

The Binary Search Tree ADT

The Binary Search Tree ADT upo_bst_t (cont'd)

EXAMPLE: Print nodes sorted by their keys to a stream

```
1 void node_fprint_visit(void *key, void *value, void *context)
2 {
3     FILE *stream = context;
4     /* Assume keys and values are integer numbers */
5     fprintf(stream, "key: %d -> value: %d\n", *((int*) key),
6             *((int*) value));
7 }
8 int main()
9 {
10     upo_bst_t tree;
11     FILE *stream = stderr;
12     ...
13     upo_bst_traverse_in_order(tree, node_fprint_visit, stream);
14     ...
15 }
```

The Binary Search Tree ADT `upo_bst_t` (cont'd)

EXAMPLE: Count the number of nodes

```
1  void node_counter_visit(void *key, void *value, void *context)
2  {
3      *((size_t*) context) += 1;
4  }
5
6  int main()
7  {
8      upo_bst_t tree;
9      size_t counter = 0;
10     ...
11     upo_bst_traverse_in_order(tree, node_counter_visit, &counter);
12     ...
13 }
```

The Binary Search Tree ADT `upo_bst_t` (cont'd)

EXAMPLE: Count the number of values equal to a given value

```
1  typedef struct {    const void *value;
2                      int (*compare)(const void*, const void*);
3                      size_t count;
4                      } visit_context_t;
5  void value_counter_visit(void *key, void *value, void *context) {
6      visit_context_t *vctx = context;
7      if (!vctx->compare(vctx->value, value)) vctx->count += 1;
8  }
9  int int_compare(const void *a, const void *b) {
10     int *ia = a, *ib = b; return (*ia > *ib) - (*ia < *ib);
11 }
12 int main() {
13     int value = 123;
14     visit_context_t visit_context;
15     upo_bst_t tree;
16     ...
17     visit_context.value = &value;
18     visit_context.compare = int_compare;
19     visit_context.count = 0;
20     upo_bst_traverse_in_order(tree, value_counter_visit, &visit_context);
21     ...
22 }
```

The Binary Search Tree ADT `upo_bst_t` (cont'd)

How to use the `upo_bst_t` ADT?

```
1  #include <upo/bst.h>
2  int int_cmp(const void *a, const void *b) {
3      const int *aa = a, *bb = b; return (*aa > *bb) - (*aa < *bb);
4  }
5  upo_bst_t bst = NULL;
6  int keys[] = {2,1,0,3};
7  char *values[] = {"two","one","zero","three"};
8  bst = upo_bst_create(int_cmp);
9  for (size_t i = 0; i < sizeof keys/sizeof keys[0]; ++i)
10     upo_bst_insert(bst, &keys[i], &values[i]);
11  int xkey = keys[2];
12  if (upo_bst_contains(bst, &xkey)) {
13     printf("Found: %d -> %s\n", xkey, *((char**) upo_bst_get(bst, &xkey)));
14     upo_bst_delete(bst, &xkey, 0);
15  } else {
16     printf("Key %d not found\n", xkey);
17  }
18  upo_bst_destroy(bst, 0);
```

Exercise #1: The BST ADT `upo_bst_t` – Part A

Tasks:

- ➊ Prepare your environment
- ➋ Implement basic operations of the BST ADT `upo_bst_t`
- ➌ Test your implementation

Exercise #1: The BST ADT `upo_bst_t` – Part A (cont'd)

Task #1 – Prepare Your Environment:

- Make sure you have already extracted the file `UP0alglib_core_toolkit.zip`
- Move to the directory `$ADT_PATH`
- Download and extract the file `bst-ex01_02-adt.zip`

```
1  include/upo/bst.h
2  src/bst_private.h
3  src/bst.c
4  test/mk/bst.mk
5  test/test_bst.c
6  test/test_bst_more.c
```

Exercise #1: The BST ADT `upo_bst_t` – Part A (cont'd)

Task #2 – Implement Basic Operations:

- Implements functions inside the file `src/bst.c` that are enclosed by comments:

```
1  /* **** EXERCISE #1 - BEGIN of FUNDAMENTAL OPERATIONS **** */  
2  ...  
3  /* **** EXERCISE #1 - END of FUNDAMENTAL OPERATIONS **** */
```

and only those that contain the comment:

```
1  /* TO STUDENTS:  
2  ...  
3  */
```

- Compile: `make clean bin`

Exercise #1: The BST ADT `upo_bst_t` – Part A (cont'd)

Task #3 – Test Your Implementation:

```
1  make clean test
2  ./test/test_bst
3  valgrind --tool=memcheck --leak-check=full ./test/test_bst
```

Exercise #1: The BST ADT `upo_bst_t` – Part A (cont'd)

Output of `./test/test_bst`

```
1 Test case 'create/destroy'... OK
2 Test case 'put/get/delete'... OK
3 Test case 'insert'... OK
4 Test case 'clear'... OK
5 Test case 'empty'... OK
6 Test case 'size'... OK
7 Test case 'height'... OK
8 Test case 'traversal'... OK
```

Exercise #1: The BST ADT `upo_bst_t` – Part A (cont'd)

Output of

```
valgrind --tool=memcheck --leak-check=full ./test/test_bst
```

```
1  ...
```

```
2
```

```
3  All heap blocks were freed -- no leaks are possible
```

```
4
```

```
5  For counts of detected and suppressed errors, rerun with: -v
```

```
6  ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from
    0)
```

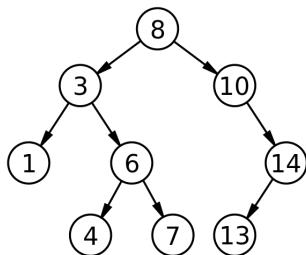
Exercise #2: The BST ADT `upb_bst_t` – Part B (cont'd)

Other common operations:

- `min(t)`: returns the smallest key in `t`
- `max(t)`: returns the largest key in `t`
- `delete_min(t)`: removes the node with the minimum key from `t`
- `delete_max(t)`: removes the node with the maximum key from `t`
- `floor(t,k)`: returns the largest key in `t` which is $\leq k$
- `ceiling(t,k)`: returns the smallest key in `t` which is $\geq k$
- `keys_range(t,l,h)`: returns the keys in `t` that are $\geq l$ and $\leq h$
- `keys(t)`: returns all the keys in `t`
- `is_bst(t,l,h)`: tells whether `t` satisfies the BST property (`l` and `h` are the possible smallest and largest values a key can take, respectively)

Exercise #2: The BST ADT `upb_bst_t` – Part B (cont'd)

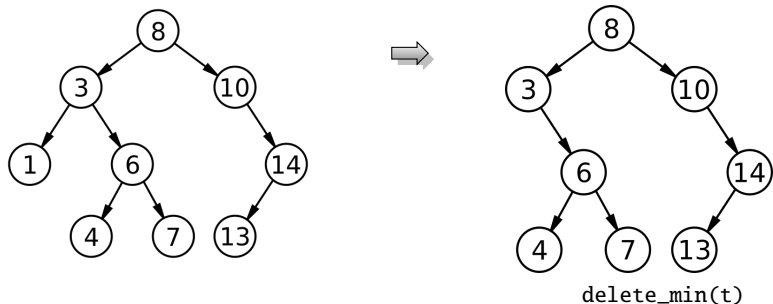
The `min(t)` and `max(t)` operations return the smallest and the largest key in the tree `t`, respectively



- `min(t) → 1`
- `max(t) → 14`

Exercise #2: The BST ADT `upb_st_t` – Part B (cont'd)

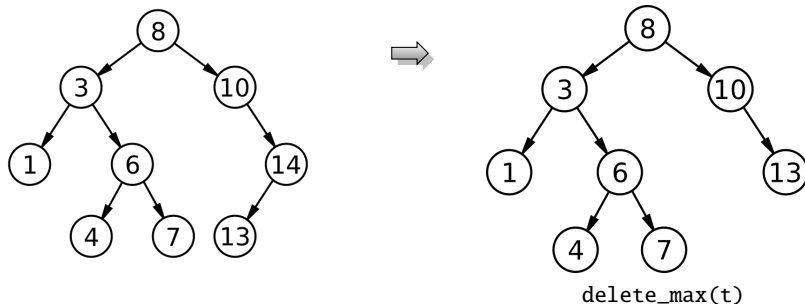
The `delete_min(t)` operation deletes the smallest key and its associated value (and the node of `t` that contains both of them)



The Binary Search Tree ADT

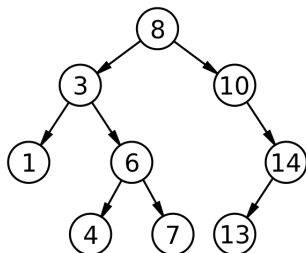
Exercise #2: The BST ADT `upo_bst_t` – Part B (cont'd)

The `delete_max()` operation deletes the largest key and its associated value (and the node of `t` that contains both of them)



Exercise #2: The BST ADT `upb_bst_t` – Part B (cont'd)

The `floor(t,k)` operation returns the largest key $\leq k$ or **null** if such key does not exist (it is not required that k is in t)

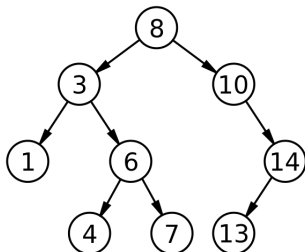


- `floor(t,8) → 8`
- `floor(t,5) → 4`
- `floor(t,13) → 13`
- `floor(t,15) → 14`
- `floor(t,0) → null`

Hint: if the key of a node n is smaller than k , it could be the floor value unless n has a right child (in such case, you first have to consider the right subtree of n).

Exercise #2: The BST ADT `up_bst_t` – Part B (cont'd)

The `ceiling(t,k)` operation returns the smallest key $\geq k$ or **null** if such key does not exist (it is not required that k is in t)

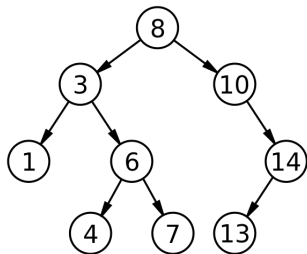


- `ceiling(t,8) → 8`
- `ceiling(t,5) → 6`
- `ceiling(t,13) → 13`
- `ceiling(t,15) → null`
- `ceiling(t,0) → 1`

Hint: similar to `floor()`, but now, when you find a node n with a key greater than k , the ceiling value could be either the key stored in n or a key stored in the left subtree of n .

Exercise #2: The BST ADT `upbost_t` – Part B (cont'd)

The `keys_range(t, l, h)` operation returns the list of keys that fall inside the range $[l, h]$, that is the keys that are $\geq l$ and $\leq h$ (it is not required that l and h are in t)

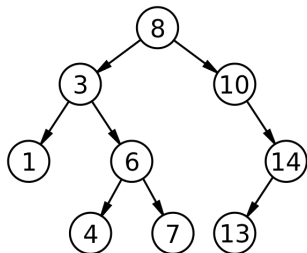


- `keys_range(t, 8, 8) → [8]`
- `keys_range(t, 3, 10) → [3, 4, 6, 7, 8, 10]`
- `keys_range(t, 6, 10) → [6, 7, 8, 10]`
- `keys_range(t, 5, 11) → [6, 7, 8, 10]`
- `keys_range(t, 0, 15) → [1, 3, 4, 6, 7, 8, 10, 13, 14]`
- `keys_range(t, 5, 5) → []` (empty list)
- `keys_range(t, 15, 20) → []` (empty list)

Hint: A possible way to solve this problem is implementing a recursive auxiliary function that takes as parameters a node of the BST (whose key has to be checked against the range $[l, h]$), the l and h values, and a pointer to the head of the list (passed by reference).

Exercise #2: The BST ADT `upo_bst_t` – Part B (cont'd)

The `keys(t)` operation returns the list of all the keys of `t`

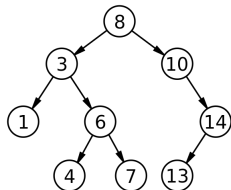


- `keys(t) → [1, 3, 4, 6, 7, 8, 10, 13, 14]`

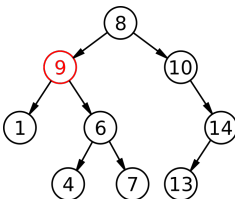
The Binary Search Tree ADT

Exercise #2: The BST ADT `upb_bst_t` – Part B (cont'd)

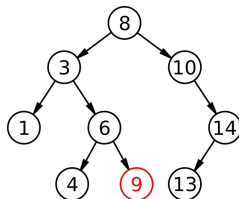
The `is_bst(t, l, h)` operation tells whether the tree t satisfies the BST property (l and h are the possible smallest and largest values a key can take, respectively)



`is_bst(t,  $-\infty$ ,  $+\infty$ )` →
true



`is_bst(t,  $-\infty$ ,  $+\infty$ )` →
false



`is_bst(t,  $-\infty$ ,  $+\infty$ )` →
false

The Binary Search Tree ADT

Exercise #2: The BST ADT `upo_bst_t` – Part B (cont'd)

The `is_bst(t, l, h)` operation... (cont'd)

Hints:

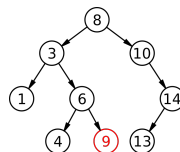
- Implement a recursive function `is_bst_impl(n, l, h)` that (1) checks if the key stored in node `n` is greater than `l` and less than `h`, and (2) recursively check its left and right subtrees.
- Set parameters `l` and `h` to suitable values for every recursive invocation of `is_bst_impl()`.

For instance, consider the BST shown in the figure.

Keys stored in the subtree rooted at node with key 3 (that is, the left child of node with key 8) cannot be greater than 8.

Similarly, keys stored in the subtree rooted at node with key 6 (that is, the right child of node with key 3) cannot be greater than 8 and also cannot be smaller than 3.

...



Exercise #2: The BST ADT `upo_bst_t` – Part B

Tasks:

- 1 Prepare your environment
- 2 Implement additional operations of the BST ADT `upo_bst_t`
- 3 Test your implementation

Exercise #2: The BST ADT `upo_bst_t` – Part B (cont'd)

Task #1 – Prepare Your Environment:

- Make sure you have already extracted the file `UP0alglib_core_toolkit.zip`
- Make sure you have already extracted the file `bst-ex01_02-adt.zip`
- Make sure you have already completed Exercise #1
- Move to the directory `$ADT_PATH`

Exercise #2: The BST ADT `upo_bst_t` – Part B (cont'd)

Task #2 – Implement Additional Operations:

- Implements functions inside the file `src/bst.c` that are enclosed by comments:

```
1  /***** EXERCISE #2 - BEGIN of EXTRA OPERATIONS *****/  
2  ...  
3  /***** EXERCISE #2 - END of EXTRA OPERATIONS *****/
```

and only those that contain the comment:

```
1  /* TO STUDENTS:  
2  ...  
3  */
```

- Compile: `make clean bin`

Exercise #2: The BST ADT `upo_bst_t` – Part B (cont'd)

Task #3 – Test Your Implementation:

```
1  make clean test
2  ./test/test_bst_more
3  valgrind --tool=memcheck --leak-check=full ./test/test_bst_more
```

Exercise #2: The BST ADT `upo_bst_t` – Part B (cont'd)

Output of `./test/test_bst_more`

- 1 Test case 'min/max'... OK
- 2 Test case 'delete min/max'... OK
- 3 Test case 'floor/ceiling'... OK
- 4 Test case 'keys range'... OK
- 5 Test case 'BST property'... OK

Exercise #2: The BST ADT `upo_bst_t` – Part B (cont'd)

Output of

```
valgrind --tool=memcheck --leak-check=full ./test/test_bst_more
```

```
1  ...
```

```
2
```

```
3  All heap blocks were freed -- no leaks are possible
```

```
4
```

```
5  For counts of detected and suppressed errors, rerun with: -v
```

```
6  ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from
    0)
```